

Lab 2: Structuring API Data with Pydantic in FastAPI

Overall Objectives:

- Students will understand the role of Pydantic in data validation and serialization.
- Students will define data schemas using Pydantic's `BaseModel`.
- Students will refactor their FastAPI application to accept structured JSON request bodies using Pydantic models.
- Students will create a `POST` endpoint that receives a Pydantic model.
- Students will create a `GET` endpoint that returns a list of Pydantic models.
- Students will observe how FastAPI automatically handles data conversion, validation, and documentation for these models.

Prerequisites:

- The `main_api.py` file from the Day 2 afternoon lab ("Building Your First API").
- VS Code with the Python 3 Dev Container running.
- FastAPI and Uvicorn installed in the container.

Part 1: Creating Pydantic Models

We will create Pydantic models that represent the `Question` data from the Olofab API we worked with earlier.

1. Import `BaseModel`:

- At the top of `main_api.py`, add an import from `pydantic`:

```
from fastapi import FastAPI
from pydantic import BaseModel # Add this import
```

2. Define the "Base" Model:

- This model will represent the data required to create a question. It doesn't have an `id` because the `id` is generated by the server.
- Add this class definition to your file (below the imports, before the `app = FastAPI()` line):

```
class QuestionBase(BaseModel):
    question: str
```

```
answer: str
```

- Explanation:
 - We inherit from `BaseModel` to get all of Pydantic's power.
 - `question: str`: We declare an attribute named `question` and use a type hint to specify it must be a string.
 - `answer: str`: Similarly, `answer` must be a string.

3. Define the "Creation" Model:

- In this simple case, the model for creating a question is the same as our base model. We can create an alias for clarity or just use `QuestionBase`. Let's create an alias for good practice.

```
class QuestionCreate(QuestionBase):  
    pass # Inherits all fields from QuestionBase
```

4. Define the "Full" Model (for Responses):

- This model represents a complete question object as it exists in our "database," including the `id`. This is what we will send back to the client in our responses.

```
class Question(QuestionBase):  
    id: int
```

- Explanation:
 - This class inherits `question` and `answer` from `QuestionBase`.
 - It adds a new required field, `id`, which must be an integer.

Part 2: Using Pydantic Models in API Endpoints

Now, let's use these models to create new `POST` and `GET` endpoints.

1. Create an In-Memory "Database":

- To make our API stateful for this lab, we'll use a simple Python list as an in-memory database.
- Add this just below your `app = FastAPI()` line:

```
# A simple in-memory database (a list of dictionaries)  
fake_db = [  
    {"id": 1, "question": "What is the airspeed velocity of an unladen swallow?", "answer": "What do you mean? An African or European swallow?"},  
    {"id": 2, "question": "What is your favorite color?", "answer":
```

```
"Blue. No, yel-- Auuuuuuuugh!"}  
]
```

2. Create a `POST` Endpoint to Add a New Question:

- Add a new path operation to your `main_api.py` file.

```
@app.post("/questions/", response_model=Question)  
async def create_question(question_to_create: QuestionCreate):  
    # In a real app, you'd save this to a database.  
    # Here, we'll add it to our fake_db list.  
  
    # Generate a new ID (simple approach for this lab)  
    new_id = max(q["id"] for q in fake_db) + 1 if fake_db else 1  
  
    # Create a full Question object including the new ID  
    new_question_data = {  
        "id": new_id,  
        "question": question_to_create.question,  
        "answer": question_to_create.answer  
    }  
  
    fake_db.append(new_question_data)  
  
    # Return the full object, which FastAPI will validate against the  
    # Question model  
    return new_question_data
```

- Explanation of Key Parts:

- `@app.post("/questions/", response_model=Question) :`
 - `response_model=Question` : This is powerful! It tells FastAPI that the response from this function should conform to the `Question` model (with an `id`). FastAPI will filter the returned data to ensure it matches this model.
- `async def create_question(question_to_create: QuestionCreate): :`
 - `question_to_create: QuestionCreate` : This is the magic! FastAPI sees this type hint and knows it should expect a JSON request body that matches the `QuestionCreate` model. It will handle all the validation and conversion for you.
 - `question_to_create` will be an actual instance of the `QuestionCreate` class, so you can access its data with dot notation (e.g., `question_to_create.question`).

3. Create a `GET` Endpoint to Read All Questions:

- Add another new path operation:

```
from typing import List # Make sure to add this import at the top

@app.get("/questions/", response_model=List[Question])
async def read_questions():
    return fake_db
```

- Explanation:
 - `from typing import List`: We need to import `List` to use it as a type hint.
 - `response_model=List[Question]`: This tells FastAPI to expect the response to be a list, where each item in the list conforms to the `Question` model.

Part 3: Testing Your Structured API

1. Run the API Server:

- Go to your integrated VS Code terminal.
- Run Uvicorn with reload enabled:

```
uvicorn main_api:app --reload
```

2. Test with the Interactive Docs (/docs):

- Open your browser to `http://127.0.0.1:8000/docs`.
- You should see your two new `/questions/` endpoints.

3. Test the `GET /questions/` Endpoint:

- Expand the `GET /questions/` endpoint.
- Click "Try it out" and then "Execute".
- Observe: The response body should be a JSON array containing the two initial questions from your `fake_db`. Notice how the documentation under "Responses" shows the exact schema for the response (`id`, `question`, `answer`).

4. Test the `POST /questions/` Endpoint:

- Expand the `POST /questions/` endpoint.
- Observe: Instead of a simple text box, the "Request body" section shows an example JSON structure based on your `QuestionCreate` model!

```
{
  "question": "string",
  "answer": "string"
}
```

- Click "Try it out".
- Modify the example JSON in the request body text area:

```
{
  "question": "What is the purpose of Pydantic?",
  "answer": "Data validation and settings management from Python type hints."
}
```

- Click "Execute".
- Observe the Response: You should get a `200 OK` response, and the response body will contain the question you just created, now including its new `id` (which should be `3`).

5. Verify the `POST`:

- Go back to the `GET /questions/` endpoint and execute it again.
- You should now see all three questions in the response, including the one you just added.

6. Test the Validation:

- Go back to the `POST /questions/` endpoint and "Try it out" again.
- This time, send an invalid request body. Try one of these:
 - Missing a field:

```
{
  "question": "This will fail"
}
```

- Wrong data type:

```
{
  "question": "Another failure",
  "answer": 12345
}
```

- Click "Execute".
- Observe the Response: You should get a `422 Unprocessable Entity` error. Look at the response body. FastAPI provides a detailed JSON response explaining exactly which field was missing or had the wrong type. This is incredibly helpful for client-side developers!

Part 4: Using the Model in a `GET` Endpoint with a Path Parameter

1. Create a `GET` Endpoint for a Single Question:

- Add this endpoint to `main_api.py`:

```
from fastapi import HTTPException # Add HTTPException to your imports

@app.get("/questions/{question_id}", response_model=Question)
async def read_single_question(question_id: int):
    # Find the question in our fake_db
    for question in fake_db:
        if question["id"] == question_id:
            return question
    # If the loop finishes without finding the question, raise an error
    raise HTTPException(status_code=404, detail="Question not found")
```

- Explanation:
 - `response_model=Question`: We specify that a successful response will be a single `Question` object.
 - `question_id: int`: FastAPI validates that the path parameter can be converted to an integer.
 - `raise HTTPException(...)`: This is the standard way in FastAPI to return HTTP error responses with a specific status code and message.

2. Test the New Endpoint:

- Save your file (Uvicorn will reload).
- Go to `/docs` and refresh.
- Test the new `/questions/{question_id}` endpoint:
 - Try it with a valid ID (e.g., `1` or `2`). You should get a `200 OK` with the question details.
 - Try it with an invalid ID (e.g., `99`). You should get a `404 Not Found` error with the detail message "Question not found".
 - Try it with a non-integer ID (e.g., `abc`). FastAPI will automatically return a `422 Unprocessable Entity` error before your function is even called!

Bonus ideas

After having completed the lab, if you still have time, you can develop the application further.

- Add all the verbs. Make sure that the verbs `DELETE`, `PUT`, `POST` and `PATCH` are all available. What is the difference between the P-verbs?
- Create a Meta-API. Let the API, for example, call OpenAI and provide an answer automatically. Or make a different endpoint, `sunny`, that just replies true if the weather is sunny where you live.